



SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the SAS entitled “**STUDENT ATTENDANCE MANGEMENT SYSTEM**” is submitted by

T. NAGA SAI CHARAN (25B21CS070)

B. L V SURYA PRAKASH (25B21CS066)

**TANATSWA HILARY
GENDERE (24B11CS529)**

BLESSED ZAMBEZI (24B11CS537)

in partial fulfillment of the requirements for the award of the B.Tech. degree in computer science and engineering for the academic year 2025-2026.

Project Guide

Dr. K. Srinu,
Associate Professor,
Department of CSE

Head of the Department

Dr. T Sudha Rani,
Associate Professor,
Department of CSE

DECLARATION

We hereby declare that the SAS entitled “**STUDENT ATTENDANCE MANGEMENT SYSTEM**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

B L V Surya Prakash	(25B21CS066)
T Naga Sai Charan	(25B21CS070)
Tanatswa Hilary Gendere	(24B11CS529)
Blessed Zambezi	(24B11CS537)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our **Student Attendance Mangement System** Project Guide, **Dr. K. Srinu, Associate Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K. Chandra Sekhar, Associate Professor** and **Dr. Hari Jyothula, Associate Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. T Sudha Rani, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Java Programming - Capstone Projects Review Schedule

Date: 29.12.2025

Capstone Project Review Schedule

Review – I (Before Pongal)

Tentative Period: On or before 05–10 January

Focus: Project Initiation and Design

- Finalization of the project problem statement and clearly defined project scope
- Completion of requirement analysis and module identification
- Preparation of preliminary database design (tables and relationships)
- Creation and initialization of the GitHub repository
- Allocation of roles and responsibilities among team members

Review – II (End of February)

Tentative Period: 20–28 February

Focus: Core Implementation & JDBC Integration

- Implementation of core project modules (Add / View / Update / Delete operations)
- Successful JDBC connectivity with database
- Execution of CRUD operations on database tables
- Menu-driven program flow implementation
- Minimum 50–60% functionality completion
- Partial working demo of the application

Review – III (End of March)

Tentative Period: 20–27 March

Focus: Final Implementation, Testing & Documentation

- Completion of all project modules
- Implementation of business logic (calculations, validations, constraints)
- End-to-end database integration and data consistency
- Comprehensive exception handling

- Final demo, documentation readiness, project video creation and viva voce preparation

ABSTRACT

The Student Attendance System (SAS) is a desktop-based application designed to efficiently manage student attendance records within an academic institution by replacing traditional manual methods with a digital solution. The system enables faculty and administrators to perform operations such as marking attendance, viewing attendance statistics, generating reports, and managing student information in a structured and secure manner using a relational database.

Developed using Core Java, Swing for the graphical user interface, JDBC for database connectivity, and SQLite as the primary backend storage (with MySQL compatibility), the application ensures data accuracy, reduces redundancy, and improves accessibility. With its role-based access control supporting Admin, Faculty, and Student roles, SHA-256 password hashing for security, and a modern dark-themed UI, the system enhances institutional productivity and provides a reliable solution for managing student attendance information.

Table of Contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	4
2.3.2 Software Requirements	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Language	5
3.1.1 Introduction to Java	5
3.2 Framework / Libraries	5
3.2.1 Introduction to Swing	5
3.3 Database Technology	5
3.3.1 JDBC (Java Database Connectivity)	6
3.3.2 SQLite Database	6
3.4 Tools & IDEs	6
3.4.1 Visual Studio Code / IntelliJ IDEA	6
4. DATABASE DESIGN	7
4.1 Database Tables	7
4.2 Table Structure	7

4.3 Keys and Constraints	8
4.4 Table Relationships	8
5. SYSTEM DESIGN	9
5.1 System Architecture	9
5.2 Modules Description	9
6. SYSTEM IMPLEMENTATION	10
7. CONCLUSION	12
8. FUTURE ENHANCEMENT	13
9. REFERENCES	14
APPENDIX (Sample Code)	15
APPENDIX (SQL Queries)	16

LIST OF FIGURES

Figure no.	Figure Title	Page no.
1	Login Screen of Student Attendance System	10
2	Admin Dashboard Screen of Student Attendance System	10
3	Add Student Screen of Student Attendance System	10
4	Mark Attendance Screen of Student Attendance System	11
5	Edit Attendance Screen of Student Attendance System	11
6	Reports Screen of Student Attendance System	11
7	Student Dashboard Screen of Student Attendance System	12

LIST OF TABLES

Table no.	Table Title	Page no.
1	Users Relational Database Table	7
2	Students Relational Database Table	7
3	Attendance Relational Database Table	8

ACRONYMS

- SAS : Student Attendance System
- JDBC : Java Database Connectivity
- GUI : Graphical User Interface
- SQL : Structured Query Language
- CRUD : Create, Read, Update, Delete
- UI : User Interface
- DAO : Data Access Object
- SHA : Secure Hash Algorithm
- OOP : Object-Oriented Programming
- JVM : Java Virtual Machine
- JAR : Java Archive
- IDE : Integrated Development Environment

1. INTRODUCTION

1.1 Brief Information about the Project

The Student Attendance System (SAS) is a desktop-based application developed to manage student attendance information efficiently within an academic institution. It provides a centralized platform to store, retrieve, update, and delete attendance records in a structured manner. The system replaces traditional manual record-keeping methods with a computerized solution, thereby improving data accuracy, reducing redundancy, and enhancing accessibility. The application is developed using Core Java for programming, Swing for designing the graphical user interface, JDBC for database connectivity, and SQLite for storing student and attendance data. It is designed to be user-friendly and suitable for use in colleges, schools, and academic institutions with three distinct user roles: Admin, Faculty, and Student.

1.2 Motivation and Contribution of Project

In many institutions, student attendance is still recorded using manual registers or basic spreadsheet tools, which leads to inefficiencies such as data duplication, difficulty in generating reports, and increased chances of errors or data loss. These limitations motivated the development of the Student Attendance System.

The key contributions of this project include:

- Providing a digital solution for managing student attendance records
- Reducing manual effort and paperwork for faculty
- Improving accuracy and consistency of attendance data
- Enabling quick report generation and attendance statistics per student
- Enhancing data security through SHA-256 password hashing and role-based access control

1.3 Objectives of the Project

The main objectives of the Student Attendance System are:

- To develop a user-friendly desktop application for managing attendance data

- To store attendance information in a structured relational database
- To provide functionalities for marking, editing, and viewing attendance records
- To enable quick searching of student details using roll number or section
- To display attendance statistics and generate downloadable reports
- To implement role-based access for Admin, Faculty, and Student users

1.4 Scope of the Project

The Student Attendance System can be used in small and medium-sized academic institutions such as engineering colleges and schools to manage student attendance records efficiently. It supports operations such as adding new student details, marking daily attendance by subject and date, editing existing attendance records, generating attendance statistics, posting and viewing announcements, and displaying all records in an organized format. The system provides a role-based interface maintaining separate dashboards for Admin, Faculty, and Student users.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Student Attendance System (SAS) must perform to effectively manage attendance-related information.

- **Mark Attendance**

The system should allow faculty to mark attendance for students by subject and date, recording each student as PRESENT or ABSENT. The system must support upsert operations so that attendance can be corrected if entered twice for the same student, subject, and date.

- **Edit Attendance**

The system should provide functionality to modify or update existing attendance records whenever required. This includes changing status from PRESENT to ABSENT or vice versa for a given student, subject, and date.

- **Add / Manage Students**

The system should allow administrators to add new student records including student name, roll number, branch, year, and section. The administrator can also view and manage existing student records.

- **View Attendance Statistics**

The system should display attendance statistics per student showing total classes held, classes attended, and percentage attendance grouped by subject. Students can view their own attendance and faculty can view statistics for their section.

- **Generate Reports**

The system should support report generation for attendance and marks. Reports should be filterable by section, year, subject, and date range.

- **Manage Announcements**

The system should allow administrators to post announcements with a title, message, and date. Students and faculty can view all posted announcements from their respective dashboards.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the Student Attendance System must satisfy.

- **Performance**

The system should respond quickly to operations like marking attendance, viewing statistics, and loading student lists without delays.

- **Security**

Only authorized users should access and modify data. Passwords must be stored as SHA-256 hashed values. Role-based access must prevent students from accessing admin or faculty-only features.

- **Usability**

The system should be simple and easy to use, allowing faculty to mark attendance and generate reports without technical difficulty.

- **Reliability**

The system should operate consistently and handle database errors without data loss. Each database connection must be created fresh per operation and closed with try-with-resources.

- **Scalability**

The system should handle increasing numbers of students and attendance records without affecting performance.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB (minimum)
- Hard Disk: 20 GB free space
- Monitor: Standard display
- Keyboard and Mouse

2.3.2 Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Programming Language: Java (JDK 11 or above)
- IDE: Visual Studio Code or IntelliJ IDEA
- Database: SQLite (bundled) or MySQL 8.0+
- Connectivity: JDBC with sqlite-jdbc or mysql-connector-j JAR

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to Java

Java is a high-level, object-oriented programming language. It is widely used for developing desktop, web, and enterprise applications due to its platform independence and robustness.

In the Student Attendance System, Java is used to implement the core functionality of the application. It handles operations such as marking attendance, loading student data, processing attendance statistics, and generating reports. Java provides features like object-oriented programming, exception handling, and strong memory management, which help in building reliable and efficient applications.

One of the main advantages of Java is that it is platform-independent, meaning the program can run on any system that has a Java Virtual Machine (JVM). This makes it suitable for developing portable applications like the Student Attendance System, which ships as a standalone executable JAR file.

3.2 Framework / Libraries

3.2.1 Introduction to Swing

Swing is a part of Java that is used to create graphical user interfaces (GUI). It provides a set of components such as buttons, labels, text fields, tables, and frames to design user-friendly applications.

In this project, Swing is used to design the user interface of the Student Attendance System. It allows users to interact with the system easily through windows such as:

- Login Window with role-based authentication
- Admin Dashboard with navigation sidebar
- Faculty Dashboard for marking and viewing attendance
- Student Dashboard for viewing own attendance statistics

- Reports Panel for attendance and marks reports
- Announcements Panel for posting and reading notices

Swing provides flexibility in designing layouts and improves the overall user experience. A custom `UITheme` class applies a consistent dark-themed color scheme across all panels.

3.3 Database Technology

The Student Attendance System uses a relational database for storing and managing attendance information in a structured manner. The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and consistency in the system. In this project, SQLite is used as the backend, and JDBC is used for connecting the Java application to the database.

3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) used in Java to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database.

In the Student Attendance System, JDBC is used to connect the Java application with the SQLite database. It performs operations such as:

- Establishing a fresh connection to the database for every operation
- Executing SQL queries (INSERT, UPDATE, DELETE, SELECT, UPSERT)
- Retrieving attendance and student data from the database
- Closing the database connection using try-with-resources

JDBC acts as a bridge between the application and the database, ensuring smooth communication and data transfer. The `DBConnection` utility class determines the database URL, driver, and credentials from environment variables or system properties, enabling seamless switching between SQLite and MySQL deployments.

3.3.2 SQLite Database

SQLite is a lightweight, serverless, self-contained relational database management system used for storing and managing structured data. It provides high performance and portability for handling attendance data in desktop applications.

In the Student Attendance System, SQLite is used to store users, students, branches, subjects, attendance records, marks, and announcements. It supports SQL for data manipulation and ensures data integrity through primary keys, foreign keys, and unique indexes. MySQL is also supported for multi-user institutional deployments via environment variable configuration.

3.4 Tools & IDEs

3.4.1 Visual Studio Code / IntelliJ IDEA

Visual Studio Code (VS Code) with Java extensions, and IntelliJ IDEA, are used as the development environments for the Student Attendance System. Both provide features such as syntax highlighting, code completion, integrated terminal, and debugging support, which help improve coding efficiency. The project includes configuration files for both IDEs and ships with build scripts (build.bat / build.sh) for compilation and JAR packaging.

4. DATABASE DESIGN

4.1 Database Tables

The Student Attendance System uses a relational database to store student and attendance information in a structured manner. The database consists of eight main tables: users, branches, faculty, subjects, students, attendance, marks, and announcements.

- The users table stores login credentials and roles for all system users.
- The students table stores student profiles and is linked to the users and branches tables.
- The attendance table records daily attendance per student per subject.
- The marks table stores exam marks per student per subject.

This separation of data helps in reducing redundancy and improves data organization.

4.2 Table Structure

4.2.1 Users Table

Field Name	Data Type	Constraint	Description
user_id	INTEGER	PK, AUTOINCREMENT	Unique user identifier
username	TEXT	NOT NULL, UNIQUE	Login username
password	TEXT	NOT NULL	SHA-256 hashed password
role	TEXT	NOT NULL	ADMIN / FACULTY / STUDENT
email	TEXT	—	Contact email address

phone	TEXT	–	Contact phone number
-------	------	---	----------------------

Table 1: Users Relational Database Table

4.2.2 Students Table

Field Name	Data Type	Constraint	Description
student_id	INTEGER	PK, AUTOINCREMENT	Unique student identifier
user_id	INTEGER	FK → users	Linked user account
student_name	TEXT	NOT NULL	Full name of student
roll_no	TEXT	NOT NULL, UNIQUE	Unique roll number
branch_id	INTEGER	FK → branches	Department / branch
year	INTEGER	–	Academic year (1–4)
section	TEXT	–	Class section (A, B, C...)

Table 2: Students Relational Database Table

4.2.3 Attendance Table

Field Name	Data Type	Constraint	Description
attendance_id	INTEGER	PK, AUTOINCREMENT	Unique record identifier
student_roll	TEXT	FK → students	Student roll number
subject	TEXT	–	Subject name

date	DATE	–	Date of attendance
status	TEXT	–	PRESENT or ABSENT

Table 3: Attendance Relational Database Table

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

• **Primary Keys:**

- user_id in users table
- student_id in students table
- attendance_id in attendance table

• **Foreign Keys:**

- user_id in students references users(user_id) ON DELETE SET NULL
- branch_id in students references branches(branch_id) ON DELETE CASCADE
- student_roll in attendance references students(roll_no) ON DELETE CASCADE

• **Unique Constraints:**

- Unique index on attendance(student_roll, subject, date) to prevent duplicate records

These constraints help in maintaining consistency and avoiding duplicate or invalid data.

4.4 Table Relationships

The students and users tables are related through the user_id field, forming a one-to-one relationship where each student account links to exactly one login user record.

- Each student belongs to one branch; one branch can have many students (one-to-many).
- Each attendance record belongs to one student; one student can have many attendance records (one-to-many).
- Each marks record belongs to one student per subject per exam type; the unique index enforces this constraint.

5. SYSTEM DESIGN

5.1 System Architecture

The Student Attendance System follows a three-layer architecture:

1. Presentation Layer (User Interface)

- Developed using Java Swing
- Provides role-specific dashboards for Admin, Faculty, and Student
- Applies a consistent dark-themed UI through the UITheme utility class

2. Application Layer (Business Logic)

- Processes user inputs through event listeners
- Performs operations like mark attendance, generate reports, manage students
- Delegates all database operations to DAO (Data Access Object) classes

3. Data Layer (Database)

- Stores student, attendance, marks, and announcement data
- Managed using SQLite (default) or MySQL (configured via environment variables)
- DBConnection utility creates a fresh JDBC connection per operation

This architecture ensures better organization, security, and maintainability.

5.2 Modules Description

The system is divided into the following modules:

1. Authentication Module

- Validates username and SHA-256 hashed password against the users table
- Routes authenticated users to Admin, Faculty, or Student dashboard based on role

2. Attendance Management Module

- Marks attendance (PRESENT/ABSENT) per student, subject, and date
- Uses upsert logic to prevent duplicate entries
- Allows editing of existing attendance records

3. Student Management Module

- Adds new student records linked to user accounts and branches
- Displays all students in a searchable and sortable JTable

4. Reports Module

- Generates attendance statistics per student showing percentage attendance by subject
- Supports filtering by section, year, subject, and date range

5. Announcements Module

- Allows admins to post announcements with title, message, and date
- Displays all announcements to all logged-in users

6. Database Connectivity Module

- Connects Java application with SQLite or MySQL via JDBC
- Determines database URL, user, and password from environment or system properties

6. SYSTEM IMPLEMENTATION

Login Screen:

Displays the login interface where the user enters username and password. Upon clicking Login, the system validates credentials against the database using SHA-256 hashed password comparison and routes the user to the appropriate role-based dashboard.

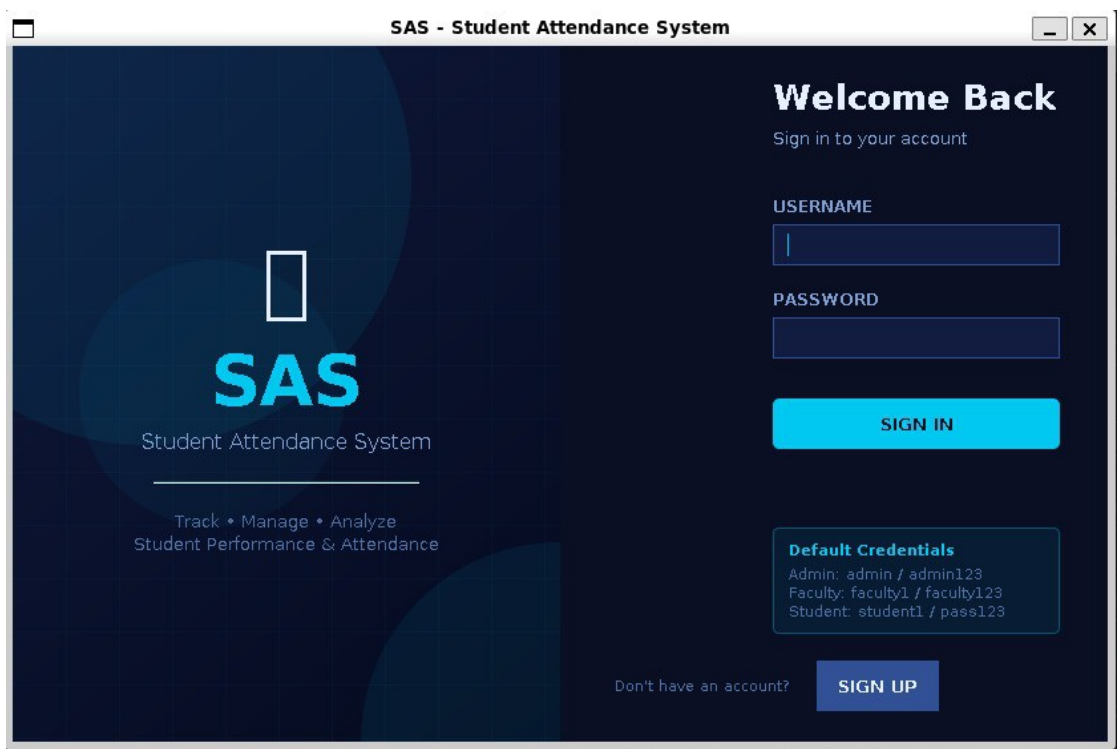


Figure 1: Login Screen of Student Attendance System

Admin Dashboard Screen:

Provides the administrator with a navigation sidebar offering options: Dashboard, Add Student, Manage Students, Mark Attendance, Edit Attendance, Reports, Announcements, Settings, and Profile.

The dashboard home shows summary statistics including total students, total attendance records, and available branches.

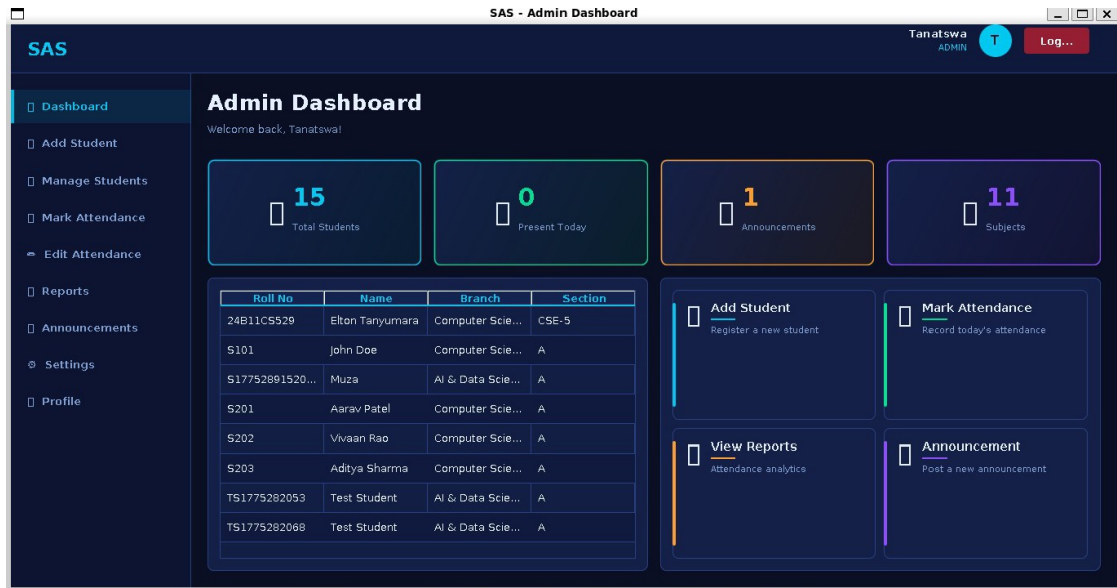


Figure 2: Admin Dashboard Screen of Student Attendance System

Add Student Screen:

Shows the form used to enter new student details including student name, roll number, branch, year, and section. The form links the student to an existing user account or creates a new one.

Add Student

Roll Number:

Name:

Branch:

Year:

Section:

Figure 3: Add Student Screen of Student Attendance System

Mark Attendance Screen:

Displays the interface where faculty selects a section, date, and subject, then marks each student as PRESENT or ABSENT. The system uses upsert logic so attendance can be corrected without duplicate errors.

The screenshot shows the 'Mark Attendance' screen in the SAS Admin Dashboard. The interface includes a sidebar with navigation options: Dashboard, Add Student, Manage Students, Mark Attendance (selected), Edit Attendance, Reports, Announcements, Settings, and Profile. The main content area has a header with 'Subject: Artificial Intelligence', 'Section: All Sections', and 'Date: 2026-04-05'. Below this are buttons for 'All Present' (green), 'All Absent' (red), and 'All Late' (blue). A summary bar shows '15 Total', '12 Present', '3 Absent', and '0 Late'. The main table lists students with columns for 'Present' (checkbox), 'Roll No', 'Student Name', and 'Status'. The table contains 8 rows of student data. At the bottom is a green 'SUBMIT ATTENDANCE' button.

Present	Roll No	Student Name	Status
☒	24B11C5529	Elton Tanyumara	PRESENT
☒	S101	John Doe	PRESENT
☐	S1775289152054	Muza	ABSENT
☒	S201	Aarav Patel	PRESENT
☒	S202	Vivaan Rao	PRESENT
☐	S203	Aditya Sharma	ABSENT
☒	TS1775282053	Test Student	PRESENT
☐	TS1775282068	Test Student	ABSENT

Figure 4: Mark Attendance Screen of Student Attendance System

Edit Attendance Screen:

Shows the interface used to modify existing attendance records. Faculty selects a roll number, subject, and date, then updates the attendance status for that record.

The screenshot shows the 'Edit Attendance' screen in the SAS Admin Dashboard. The sidebar is the same as in Figure 4, with 'Edit Attendance' selected. The main content area has a header with 'Subject: Artificial Intelligence' and 'Date: 2026-04-05', followed by a 'Load Records' button. A summary bar shows '3 Present', '12 Absent', and '0 Late'. The main table lists students with columns for 'Roll No', 'Student Name', and 'Current Status'. The table contains 9 rows of student data. At the bottom is a blue 'SAVE CHANGES' button.

Roll No	Student Name	Current Status
S203	Aditya Sharma	ABSENT
TS1775282053	Test Student	ABSENT
TS1775282068	Test Student	ABSENT
TS1775301390	Test Student	PRESENT
TS1775301420	Test Student	ABSENT
TS1775301510	Test Student	PRESENT
TS1775302073	Test Student	ABSENT
TS1775329058	Test Student	PRESENT
TS1775329107	Test Student	ABSENT

Figure 5: Edit Attendance Screen of Student Attendance System

Reports Screen:

Displays attendance statistics per student showing subject-wise total classes, attended count, and percentage. Reports can be filtered by section and year for administrative review.

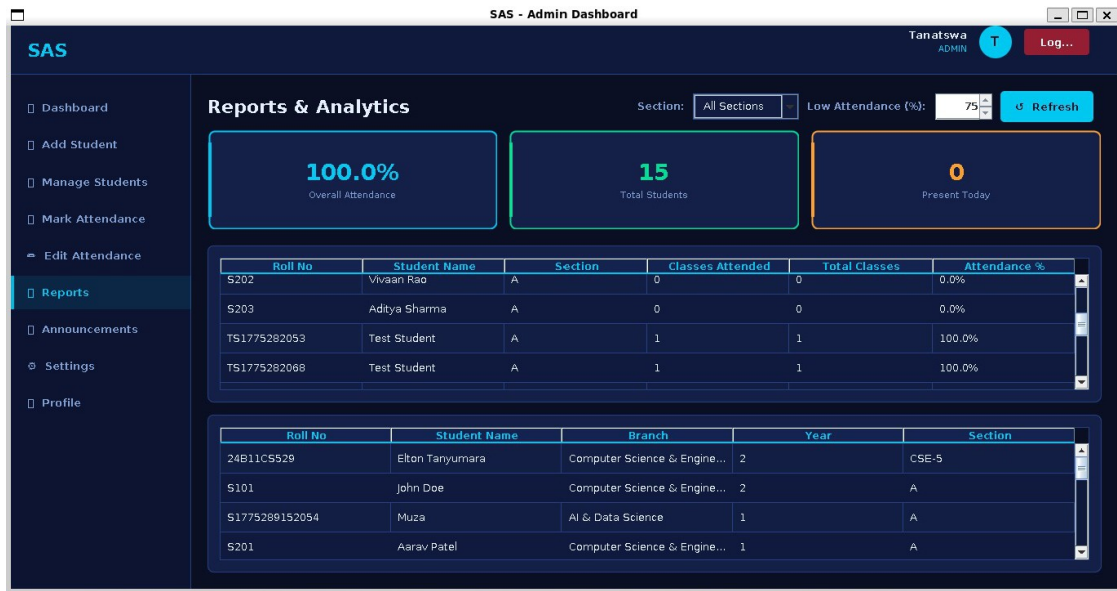


Figure 6: Reports Screen of Student Attendance System

Student Dashboard Screen:

Provides students with a personal view of their own attendance statistics organized by subject. Students can also view announcements and update their profile information from this dashboard.

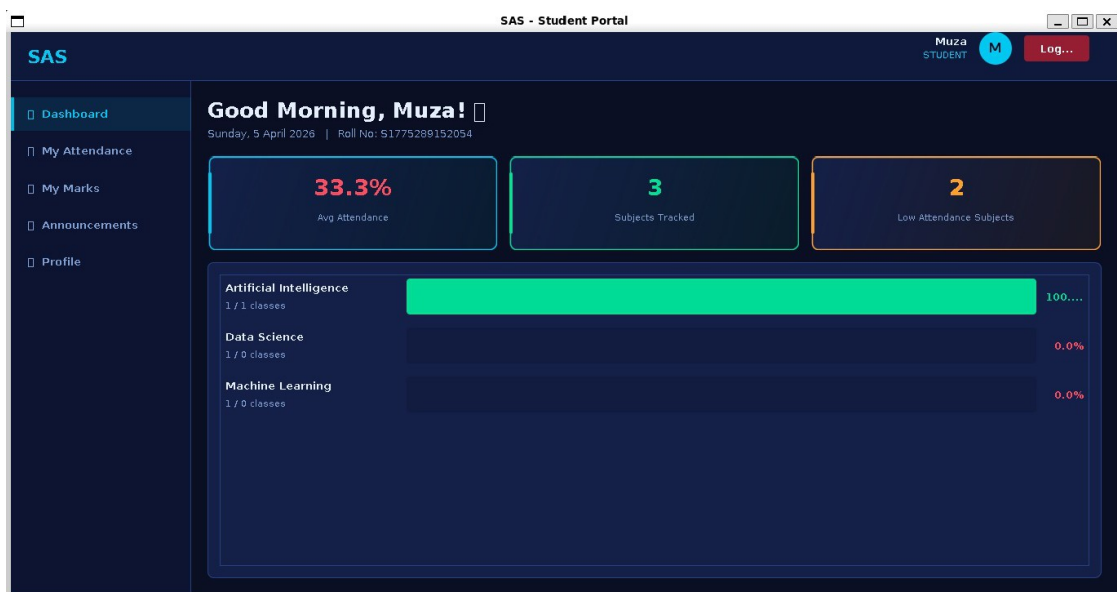


Figure 7: Student Dashboard Screen of Student Attendance System

7. CONCLUSION

The Student Attendance System has been successfully developed to manage student attendance information in an efficient and organized manner. The system replaces manual record-keeping with a computerized solution, thereby reducing errors, saving time, and improving data accuracy.

The application provides essential functionalities such as marking attendance, editing records, viewing statistics, generating reports, and managing student information through a user-friendly interface. By using technologies like Java, Swing, JDBC, and SQLite, the system ensures reliable performance, secure data handling through SHA-256 password hashing, and easy access to attendance information.

Overall, the system improves the efficiency of managing attendance data and reduces the workload of faculty and administrative staff. It also provides a scalable solution that can be further enhanced in the future by adding features such as web-based access, automated notifications, and biometric integration.

In conclusion, the Student Attendance System meets the project objectives and serves as an effective tool for managing student attendance records in academic institutions. The system is deployable as a standalone JAR file, making it portable and easy to run on any Java-enabled machine.

8. FUTURE ENHANCEMENT

The Student Attendance System developed in this project provides basic functionalities for managing student attendance records. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.

Some possible future enhancements include:

- **Web-Based Application:**

Converting the desktop application into a web-based system for remote access, allowing faculty to mark attendance from any browser without installing the desktop application.

- **Automated Email / SMS Notifications:**

Sending automated alerts to students and parents when attendance falls below the required threshold (e.g., below 75%).

- **Biometric / QR Code Integration:**

Integrating fingerprint scanners or QR code-based attendance marking to further eliminate manual data entry errors.

- **Multi-User Network Support:**

Upgrading the database backend to a dedicated MySQL server to allow multiple faculty members to mark attendance simultaneously from different machines.

- **Improved Reporting & Analytics:**

Adding graphical charts and trend analysis for attendance patterns, exportable to PDF or Excel format for institutional reporting.

These enhancements can make the system more scalable, secure, and suitable for real-time institutional use.

9. REFERENCES

The following references were used during the development of the Student Attendance System:

1. Herbert Schildt, Java: The Complete Reference, Oracle Press, 11th Edition.
2. E. Balagurusamy, Programming with Java, McGraw Hill Education.
3. Korth, Silberschatz, Sudarshan, Database System Concepts, McGraw Hill.
4. SQLite Documentation, <https://www.sqlite.org/docs.html>
5. Java Documentation (Oracle), <https://docs.oracle.com/javase>
6. JDBC Tutorial – GeeksforGeeks, <https://www.geeksforgeeks.org/jdbc/>
7. Java Swing Tutorial – TutorialsPoint,
https://www.tutorialspoint.com/java_swing
8. W3Schools Java Tutorial, <https://www.w3schools.com/java/>

APPENDIX

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the Java application and the SQLite database using JDBC.

DBConnection.java:

```
public class DBConnection {
    private static final String URL =
        "jdbc:sqlite:absents.db?foreign_keys=on";

    public static synchronized Connection getConnection()
        throws SQLException {
        try {
            Class.forName("org.sqlite.JDBC");
            return DriverManager.getConnection(URL);
        } catch (ClassNotFoundException e) {
            throw new SQLException("Driver not found", e);
        }
    }
}
```

Mark Attendance (Upsert) Code:

This code marks or updates attendance using SQLite's ON CONFLICT upsert syntax.

AttendanceDAO.java:

```
public boolean markAttendance(String rollNo,
    String subject, String date, String status) {
    String sql = "INSERT INTO attendance" +
        "(student_roll,subject,date,status) VALUES(?,?,?,?) " +
        "ON CONFLICT(student_roll,subject,date) " +
        "DO UPDATE SET status=excluded.status";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, rollNo);
        ps.setString(2, subject);
        ps.setString(3, date);
        ps.setString(4, status);
        return ps.executeUpdate() > 0;
    }
}
```

A.2 SQL Queries

Create Users Table:

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT NOT NULL UNIQUE,  
    password TEXT NOT NULL,  
    role TEXT NOT NULL,  
    email TEXT,  
    phone TEXT  
);
```

Create Attendance Table:

```
CREATE TABLE attendance (  
    attendance_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_roll TEXT,  
    subject TEXT,  
    date DATE,  
    status TEXT,  
    FOREIGN KEY (student_roll)  
        REFERENCES students(roll_no) ON DELETE CASCADE  
);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS unique_attendance  
    ON attendance(student_roll, subject, date);
```